

Build Tools & Git: Exam Prep Quiz

12 questions · answer key and explanations at the end

1. Which phases run when you execute `mvn package`?

- a) package only
- b) validate, compile, test, package
- c) compile, package, install
- d) validate, compile, test, package, verify, install, deploy

2. What is the difference between `./gradlew build` and `./gradlew assemble`?

- a) assemble also runs the tests; build does not
- b) They are identical
- c) build compiles, tests and packages; assemble packages without running tests
- d) assemble deletes the build/ directory

3. What is the difference between Gradle's `implementation` and `api`?

- a) `api` exposes the dependency to modules that consume this module; `implementation` does not
- b) `implementation` is only for tests
- c) `api` is available only at runtime
- d) There is no difference

4. Which pair is correct?

- a) `compileOnly`: runtime only. `runtimeOnly`: compile time only.
- b) `compileOnly`: not included at runtime. `runtimeOnly`: not available at compile time.
- c) Both are available everywhere.
- d) Both are available only for tests.

5. What does Maven's `<scope>provided</scope>` mean?

- a) Available only for tests
- b) Available everywhere (the default)
- c) Compile-time only; supplied by the runtime environment
- d) Used only during annotation processing

6. Why should you use the Gradle wrapper (`./gradlew`) instead of a system-wide Gradle?

- a) It is faster to type
- b) It pins one Gradle version so every contributor builds the same way
- c) It removes the need for `build.gradle`
- d) It automatically commits the build output

7. Why should `build/` or `target/` NOT be committed to version control?

- a) Git cannot store folders
- b) They contain the source code
- c) They hold generated output that can be rebuilt; they belong in `.gitignore`
- d) They are required by the wrapper

8. What is the difference between `git fetch` and `git pull`?

- a) `fetch` downloads remote changes without merging; `pull` = `fetch` + `merge`
- b) `fetch` uploads changes; `pull` downloads them
- c) They do exactly the same thing
- d) `pull` only works on the first clone

9. Which `git reset` mode keeps your changes STAGED?

- a) `--soft`
- b) `--mixed`
- c) `--hard`
- d) None of them

10. You already pushed a bad commit to a shared branch. What is the safe way to undo it?

- a) `git reset --hard`, then `git push -f`
- b) `git rebase`
- c) `git revert`
- d) `git commit --amend`

11. Which statement about merge vs. rebase is correct?

- a) Merge rewrites history; rebase does not
- b) Merge creates a merge commit and keeps original commits; rebase replays commits on top for linear history but rewrites them
- c) Rebase is always safe on any branch
- d) Merge produces linear history

12. What causes a merge conflict, and how is it resolved?

- a) Two branches changed the same part of a file differently; you fix the marked sections manually, then commit
- b) The `.gitignore` is missing; Git fixes it automatically
- c) Two branches have the same name; delete one
- d) A commit is too large; use `--amend`

Answer key

1. Which phases run when you execute `mvn package`?

Answer: b) validate, compile, test, package

Maven's lifecycle is a fixed sequence, and running a phase runs every phase before it. So package runs validate, compile, test and package.

2. What is the difference between `./gradlew build` and `./gradlew assemble`?

Answer: c) build compiles, tests and packages; assemble packages without running tests

build = compile + test + package. assemble skips the tests. clean is the one that deletes build/.

3. What is the difference between Gradle's `implementation` and `api`?

Answer: a) `api` exposes the dependency to modules that consume this module; `implementation` does not

Both are available everywhere in the module, but `api` also exposes the dependency to consumers of the module.

4. Which pair is correct?

Answer: b) `compileOnly`: not included at runtime. `runtimeOnly`: not available at compile time.

`compileOnly` is needed to compile but is not packaged or available at runtime. `runtimeOnly` is the reverse.

5. What does Maven's `<scope>provided</scope>` mean?

Answer: c) Compile-time only; supplied by the runtime environment

`provided` is for compile time only, because the environment supplies it at runtime. `scope=test` is tests only, and with no scope the default is compile.

6. Why should you use the Gradle wrapper (`./gradlew`) instead of a system-wide Gradle?

Answer: b) It pins one Gradle version so every contributor builds the same way

The wrapper downloads and uses a pinned Gradle version, so builds behave the same for everyone.

7. Why should `build/` or `target/` NOT be committed to version control?

Answer: c) They hold generated output that can be rebuilt; they belong in `.gitignore`

These directories are generated, machine-specific output. Committing them bloats the repo and causes needless conflicts.

8. What is the difference between `git fetch` and `git pull`?

Answer: a) `fetch` downloads remote changes without merging; `pull` = `fetch` + `merge`

`fetch` updates the remote-tracking ref (e.g. `origin/master`) and leaves your files alone. `pull` then also merges (or rebases with `--rebase`) into your current branch.

9. Which `git reset` mode keeps your changes STAGED?

Answer: a) `--soft`

`--soft` keeps changes staged. `--mixed` (the default) keeps them in the working tree but unstaged. `--hard` discards them.

10. You already pushed a bad commit to a shared branch. What is the safe way to undo it?

Answer: c) `git revert`

`revert` adds a new commit that undoes the old one and leaves history intact, so it pushes normally. `reset` rewrites history and the remote rejects it unless forced.

11. Which statement about merge vs. rebase is correct?

Answer: b) Merge creates a merge commit and keeps original commits; rebase replays commits on top for linear history but rewrites them

Merge is non-destructive but can clutter history. Rebase gives cleaner linear history but creates new commits, so never rebase shared branches like main.

12. What causes a merge conflict, and how is it resolved?

Answer: a) Two branches changed the same part of a file differently; you fix the marked sections manually, then commit

Git marks the conflicting sections and pauses the merge. You edit the file to the result you want, stage it, and commit to finish.